# Moving an object in PyGame

25 January 2026 16:58

To move an object such that it moves horizontally when pressing the right arrow key or left arrow key on the keyboard and it moves vertically when pressing up arrow key or down arrow key.
The main concept of doing this by changing the coordinates of the object and refreshing the screen. When the screen refreshes, every time window color gets filled with original color and a new rectangle is formed. So when arrow key gets pressed, coordinate changes and it appears that the object is moving.

**Change in Co-ordinates for respective keys pressed:**
Left arrow key: Decrement in X coordinate.
Right arrow key: Increment in X coordinate.
Up arrow key: Decrement in Y coordinate.
Down arrow key: Increment in Y coordinate.

```
import pygame
pygame.init()
win = pygame.display.set_mode((500,500))
pygame.display.set_caption("Moving rectangle")
# object current co-ordinates
x = 200
y = 200
# dimension of the object
width = 20
height = 20
# velocity/speed of movement
vel = 10
while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()

        # stores keys pressed
        keys = pygame.key.get_pressed()

        # if left arrow key is pressed
        if keys[pygame.K_LEFT] and x > 0:
            x -= vel   #decrement the x co-ordinate

        # if right arrow key is pressed
        if keys[pygame.K_RIGHT] and x < 500 - width:
            x += vel # increment in x co-ordinate

        #if up arrow key is pressed
        if keys[pygame.K_UP] and y > 0:
            y -= vel # deccrement the value of y co-ordinate

        # if down arrow key is pressed
        if keys[pygame.K_DOWN] and y < 500 - height:
```

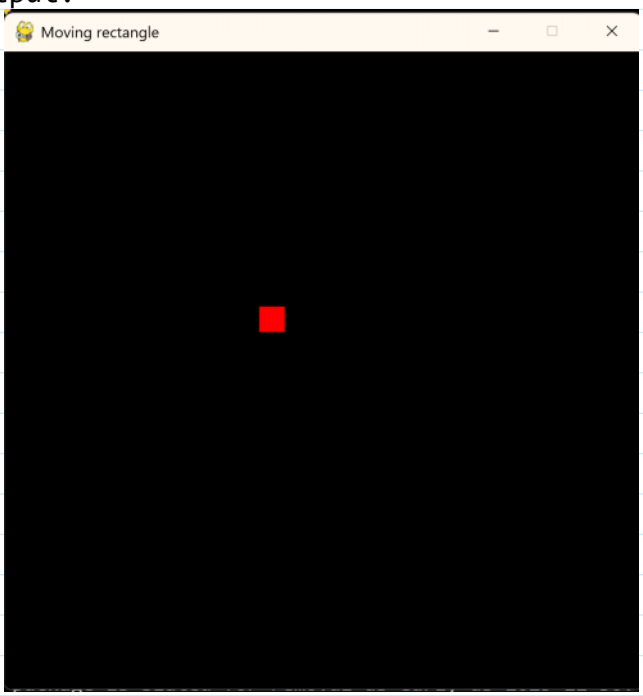
```
            y += vel    #increment the value of y co-ordinate

        # Completely fill the surface object with black color.
        win.fill((0,0,0))

        # Drawing object on screen which is a rectangle here.
        pygame.draw.rect(win,(255,0,0),(x,y,width, height))

        # It refreshes the window.
        pygame.display.update()
```

Output:




**Making an object jump in pygame**

The basic formula from classical mechanics to make an object jump.

F = 1/2 * m * v^2

Where F is the force up or down. M is the mass of the object and V is the velocity. The velocity goes overtime because when the object jumps, the velocity will not increase more in this simulation. Where an object reaches the ground, the jump ends.

If isjump variable is True or False it indicates object is jumping or not. If isjump is True, object position will be updated according to above formula.

```
import pygame
pygame.init()
win = pygame.display.set_mode((500,500))
pygame.display.set_caption("Jump Game")
x = 200
y = 200
width = 30
```

```python
height = 40
isjump = False
v = 5
m = 1
while True:
    win.fill((0,0,0))
    pygame.draw.rect(win, (255,0,0),(x,y,width,height))

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()

        #stores key pressed
        keys = pygame.key.get_pressed()

        if isjump == False:
            # if space bar is pressed
            if keys[pygame.K_SPACE]:
                # make isjump equal to True
                isjump = True

        if isjump:
            # calculate force(F) = 1/2*mass*velocity^2
            F = (1/2)*m*(v**2)

            # change in the y co-ordinate
            y -= F

            # Decreasing velocity while going up and become negative while coming down.
            v -= 1

            # object reached its maximum height
            if v < 0:
                # Negative sign is added to counter negative velocity.
                m = -1
            # Object reaches its original state
            if v == -6:
                isjump = False

                # Setting original values to v and m.
                v = 5
                m = 1

        pygame.time.delay(10)
        pygame.display.update()
```

Homework:

**Question 1: Keyboard-Controlled Player Movement with Boundary Constraints**
Write a Python program using the **Pygame** library to create a window of size **600 × 600 pixels.** Inside the window, display a **blue rectangular player object** initially positioned at the center of the screen.

The program should meet the following requirements:

a. The rectangle should move **left, right, up, and down** using the **arrow keys.**
b. Each key press should move the rectangle by **10 pixels** in the respective direction.
c. The rectangle **must not move outside the window boundaries.**
d. The background color of the window should be **black**, and the screen should refresh smoothly after each movement.
e. Use appropriate Pygame functions to:
   - Detect keyboard input
   - Update the object's coordinates
   - Redraw the object after clearing the screen
f. Properly handle the **QUIT event** so the window closes gracefully.

**Expected Learning Outcome:**
Understanding of coordinate-based movement, keyboard event handling, screen refreshing, and boundary checking in Pygame.

**Question 2: Horizontal Movement with Vertical Jump Simulation**
Design a Python program using **Pygame** that simulates a **character movement with jumping ability.**

Program specifications:

a. Create a window of size **500 × 500 pixels.**
b. Display a **rectangular object** that can:
   - Move **left and right** using the **LEFT and RIGHT arrow keys.**
   - **Jump vertically** when the **SPACE bar** is pressed.
c. Implement the jump using the following logic:
   - Use a Boolean variable isJump to track whether the object is currently jumping.
   - Use velocity (v) and mass (m) variables.
   - Apply the formula:
     [
     F = \frac{1}{2} \times m \times v^2
     ]
   - Decrease the velocity gradually to simulate upward motion and gravity-like downward motion.
d. Ensure the object returns to its **original vertical position** after the jump.
e. Prevent multiple jumps while the object is already in the air.
f. Continuously refresh the screen to animate the jump smoothly.

**Expected Learning Outcome:**
Application of basic physics concepts to animation, Boolean state management, and smooth motion simulation in games.

**Question 3: Controlled Player Movement with Speed and Jump Limits**
Create a Python program using **Pygame** to develop a simple **player control system** with movement and jumping features.

The program must satisfy the following conditions:

a. Create a window of size **700 × 500 pixels.**
b. Draw a rectangular player object that:
   - Moves **left and right** using arrow keys.
   - Moves **up and down** using arrow keys.
   - Performs a **jump** using the **SPACE bar.**
c. Introduce a **speed variable** to control movement velocity.
d. During the jump:
   - Use a velocity variable that decreases each frame.
   - Stop the jump when the object reaches the ground.
e. The object should:

- Not cross the window boundaries.
- Not initiate another jump while already jumping.

f. Maintain a constant frame delay to ensure smooth animation.

g. Use comments to explain each major section of the code.

**Expected Learning Outcome:**

Comprehensive understanding of game loops, coordinate updates, velocity control, jump mechanics, and structured Pygame programming.